

Electro Pi — QA Automation Engineer Technical Assessment

Candidate: Mohamed El Nomrosy

Position: QA Automation Engineer

Assessment: Technical Assessment

Date: September 2026

GitHub Repository: <https://github.com/mohamednomrosy/electropi-qa-automation>

Assumptions

The assessment does not provide a live application environment, application URL, authentication endpoint, or database schema. Therefore, selectors, URLs, API authentication details, HTTP status codes, and database structures used in this submission are based on reasonable assumptions and should be replaced or confirmed when the actual environment is available.

Therefore:

- The UI URL and selectors are assumed and should be replaced with actual application values when the environment is available.
- API error status codes are assumed based on common REST API conventions.
- The authentication endpoint and token response structure are assumed.
- Database table and column names are assumed based on the assessment requirements.
- No claim is made that the UI automation was executed against a live Electro Pi environment.

Part 1 — UI Test Automation & Architecture

1. Framework Choice

Selected Framework: Playwright with Java

I would choose **Playwright with Java** for this project.

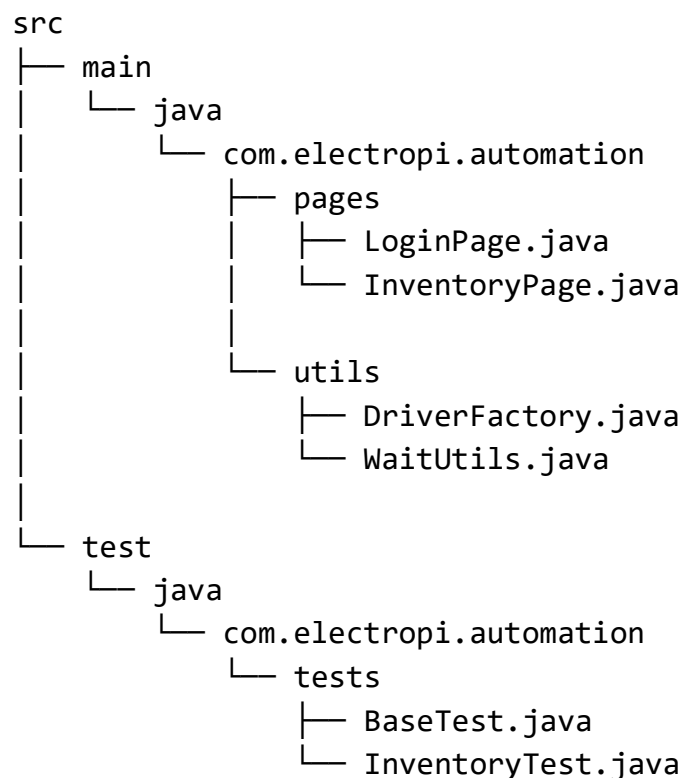
The application is a multi-tenant, cloud-based POS and inventory management system that relies heavily on dynamic UI components and asynchronous operations. Playwright is well suited for this type of application because it provides:

- Built-in auto-waiting for elements and actions.
- Reliable handling of dynamic and asynchronous UI components.
- Support for Chromium, Firefox, and WebKit.
- Network request and response interception.
- Easy handling of multiple pages, tabs, and browser contexts.
- Parallel test execution.
- Automatic screenshots, videos, and traces for debugging.
- Strong support for modern web applications.
- Good integration with Maven, TestNG, CI/CD pipelines, and reporting tools.

Playwright's automatic waiting mechanisms can significantly reduce flaky tests compared with approaches that rely heavily on fixed delays.

2. Design Pattern and Project Structure

I would use the **Page Object Model (POM)** combined with reusable utility and test layers.



Page Object Model

Each page contains its own locators and user actions.

- LoginPage → Login-related elements and actions.
- InventoryPage → Inventory navigation, product creation, and validation.
- BaseTest → Browser setup and teardown.
- DriverFactory → Browser initialization and configuration.
- WaitUtils → Reusable synchronization methods.

Benefits

This structure improves:

- **Maintainability:** UI changes can be handled within the corresponding page object.
- **Reusability:** Common actions can be reused across multiple tests.
- **Readability:** Tests focus on business scenarios rather than implementation details.
- **Scalability:** Additional modules such as Orders, Customers, Reports, and Payments can be added without duplicating code.
- **Debugging:** Page-specific failures are easier to identify and troubleshoot.

3. Implementation

LoginPage.java

```
package com.electropi.automation.pages;

import com.microsoft.playwright.Page;

public class LoginPage {

    private final Page page;

    private final String emailInput = "[data-testid='email']";
    private final String passwordInput = "[data-testid='password']";
    private final String loginButton = "[data-testid='login-button']";

    public LoginPage(Page page) {
        this.page = page;
    }
}
```

```

    public void navigateToLoginPage() {
        page.navigate("https://pos.example.com/login");
    }

    public void login(String email, String password) {
        page.locator(emailInput).fill(email);
        page.locator(passwordInput).fill(password);
        page.locator(loginButton).click();
    }
}

```

InventoryPage.java

```

package com.electropi.automation.pages;

import com.microsoft.playwright.Page;

public class InventoryPage {

    private final Page page;

    private final String inventoryMenu =
        "[data-testid='inventory-menu']";

    private final String productNameInput =
        "[data-testid='product-name']";

    private final String productPriceInput =
        "[data-testid='product-price']";

    private final String saveButton =
        "[data-testid='save-product']";

    private final String successToast =
        "[data-testid='success-toast']";

    public InventoryPage(Page page) {
        this.page = page;
    }
}

```

```

    public void navigateToInventory() {
        page.locator(inventoryMenu).click();
        page.waitForURL("**/inventory");
    }

    public void createProduct(String productName, String price) {
        page.locator(productNameInput).fill(productName);
        page.locator(productPriceInput).fill(price);
        page.locator(saveButton).click();
    }

    public boolean isSuccessToastDisplayed() {
        return page.locator(successToast).isVisible();
    }

    public String getSuccessToastMessage() {
        return page.locator(successToast).innerText();
    }
}

```

DriverFactory.java

```

package com.electropi.automation.utils;

import com.microsoft.playwright.Browser;
import com.microsoft.playwright.BrowserType;
import com.microsoft.playwright.Playwright;

public class DriverFactory {

    private static Playwright playwright;
    private static Browser browser;

    public static Browser launchBrowser() {

        playwright = Playwright.create();

        browser = playwright.chromium().launch(
            new BrowserType.LaunchOptions()
                .setHeadless(true)
        );
    }
}

```

```

        return browser;
    }

    public static void closeBrowser() {

        if (browser != null) {
            browser.close();
        }

        if (playwright != null) {
            playwright.close();
        }
    }
}

```

BaseTest.java

```

package com.electropi.automation.tests;

import com.electropi.automation.utils.DriverFactory;
import com.microsoft.playwright.Browser;
import com.microsoft.playwright.BrowserContext;
import com.microsoft.playwright.Page;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public class BaseTest {

    protected Browser browser;
    protected BrowserContext context;
    protected Page page;

    @BeforeMethod
    public void setUp() {

        browser = DriverFactory.launchBrowser();

        context = browser.newContext();

        page = context.newPage();
    }
}

```

```

    }

    @AfterMethod
    public void tearDown() {

        if (context != null) {
            context.close();
        }

        DriverFactory.closeBrowser();
    }
}

```

InventoryTest.java

```

package com.electropi.automation.tests;

import com.electropi.automation.pages.InventoryPage;
import com.electropi.automation.pages.LoginPage;
import org.testng.Assert;
import org.testng.annotations.Test;

public class InventoryTest extends BaseTest {

    @Test
    public void storeAdminCanCreateInventoryItem() {
        LoginPage loginPage = new LoginPage(page);
        InventoryPage inventoryPage = new InventoryPage(page);

        loginPage.navigateToLoginPage();

        loginPage.login(
            "storeadmin@example.com",
            "Password123"
        );

        inventoryPage.navigateToInventory();

        inventoryPage.createProduct(

```

```

        "Wireless Mouse",
        "25.00"
    );

    Assert.assertTrue(
        inventoryPage.isSuccessToastDisplayed(),
        "Success toast should be displayed after saving the
product."
    );

    Assert.assertTrue(
        inventoryPage.getSuccessToastMessage()
            .toLowerCase()
            .contains("success"),
        "Success toast should contain a success message."
    );
}
}

```

Note: The credentials, URL, and selectors above are placeholders because no live environment was provided. In a real implementation, credentials would be stored securely using environment variables or CI/CD secrets rather than being hardcoded.

4. Handling Dynamic Elements and Preventing Flaky Tests

To improve test stability, I would avoid fixed delays such as:

```
Thread.sleep(5000);
```

Instead, the framework should wait for meaningful application states.

A. Use Stable Selectors

Prefer stable attributes such as:

```

data-testid
data-qa
id

```

For example:

```
page.locator("[data-testid='save-product']");
```

This is more maintainable than relying on long XPath or CSS paths based on the UI structure.

B. Use Playwright Auto-Waiting

Playwright automatically waits for elements to become actionable before performing actions such as:

```
click();  
fill();  
check();
```

C. Wait for Specific Application States

```
page.locator("[data-testid='success-toast']")  
    .waitFor();
```

This is preferable to waiting for an arbitrary amount of time.

D. Wait for Navigation

```
page.waitForURL("**/inventory");
```

E. Synchronize with API Responses

```
page.waitForResponse(  
    response -> response.url()  
        .contains("/api/v1/inventory/items")  
        && response.status() == 201,  
    () -> {  
        page.locator("[data-testid='save-product']").click();  
    }  
);
```

F. Avoid Test Dependencies

Each test should be independently executable whenever possible.

Required test data can be:

- Created through APIs.

- Generated dynamically.
- Loaded from test data files.
- Reset before or after execution.

G. Use Test Isolation

For a multi-tenant application, tests should ensure that data belonging to one store or tenant does not affect another tenant.

Each test should use an appropriate:

- Tenant/store context.
- User role.
- Test data set.
- Browser context/session.

H. Capture Debugging Artifacts

For failed tests, I would configure the framework/CI pipeline to capture:

- Screenshots.
- Playwright traces.
- Browser console logs.
- Network information where required.
- Test execution logs.

Part 2 — API Testing Strategy

API Endpoint

Method: POST

Endpoint:

`/api/v1/inventory/items`

Headers:

Authorization: Bearer <token>
Content-Type: application/json

Request Body:

```
{  
  "item_name": "Wireless Mouse",  
  "sku": "MS-001",  
  "quantity": 50,  
  "price": 25.00,  
  "category_id": 3  
}
```

Assumption: The provided API specification does not define the exact HTTP status codes for validation, authentication, and duplicate-data errors. Therefore, 400, 401, and 409 are assumed based on common REST API conventions and should be confirmed against the actual backend contract.

1. Positive Test Cases

API-01 — Create Inventory Item with Valid Data

Input:

- item_name: Wireless Mouse
- sku: MS-001
- quantity: 50
- price: 25.00
- category_id: 3
- Valid authentication token.

Expected Status Code:

201 Created

Expected Result:

The inventory item is successfully created and the response contains the submitted SKU MS-001.

API-02 — Create Another Inventory Item with Valid Data

Input:

Valid inventory item data with a different unique SKU.

Expected Status Code:

201 Created

Expected Result:

A new inventory item is successfully created and the response contains the correct item details and SKU.

2. Negative Test Cases

API-03 — Invalid or Missing Authentication

Input:

Missing or invalid Authorization token.

Expected Status Code:

401 Unauthorized

Expected Result:

The request is rejected and no inventory item is created.

API-04 — Missing Required Field

Input:

Remove `item_name` from the request body.

Expected Status Code:

400 Bad Request

Expected Result:

The request is rejected with a validation error indicating that the required field is missing.

API-05 — Duplicate SKU

Input:

Use an SKU that already exists, such as MS-001.

Expected Status Code:

409 Conflict

Expected Result:

The request is rejected and a duplicate inventory item is not created.

3. Postman Automation — Dynamic Token Generation

The authentication token should not be hardcoded in the inventory request.

A separate authentication request can be executed before the inventory API request. The returned token can then be stored dynamically in a Postman environment variable named `authToken`.

```
const response = pm.response.json();  
  
pm.environment.set("authToken", response.token);
```

The inventory request can then use:

Authorization: Bearer {{authToken}}

Assumption: An authentication endpoint is available and returns the token in a property named `token`. The exact endpoint and response property should be updated according to the actual authentication API contract.

4. Postman Tests Tab

The following script validates that the API returns 201 Created and that the returned SKU matches the submitted SKU.

```
pm.test("Status code is 201 Created", function () {  
    pm.response.to.have.status(201);  
});
```

```
const response = pm.response.json();
```

```
pm.test("SKU is returned correctly", function () {  
    pm.expect(response.sku).to.eql("MS-001");  
});
```

If the API response wraps the item inside a data object:

```
pm.expect(response.data.sku).to.eql("MS-001");
```

5. Additional API Validations

For a more comprehensive API automation strategy, I would also validate:

- Response body structure.
- SKU is not empty.
- Item name matches the request.
- Quantity matches the request.
- Price matches the request.
- Category ID is correct.
- Response time is within the agreed SLA.
- No unexpected sensitive information is returned.
- Appropriate error messages are returned for invalid requests.
- The created item can subsequently be retrieved using its SKU or ID.

Part 3 — CI/CD & Database Validation

Assessment Requirement 1 — Database Validation

Requirement

After the API creates an inventory item, provide a SQL query to retrieve the item from the products table and join the categories table to verify that the item belongs to the **Electronics** category.

SQL Query

Assuming:

- products contains the inventory items.
- categories contains product categories.
- products.category_id references categories.id.
- categories.name contains the category name.

```
SELECT
    p.id,
    p.item_name,
    p.sku,
    p.quantity,
    p.price,
    c.name AS category_name
FROM products p
JOIN categories c
    ON p.category_id = c.id
WHERE p.sku = 'MS-001'
    AND c.name = 'Electronics';
```

Expected Result

The query should return:

Item Name: Wireless Mouse

SKU: MS-001

Quantity: 50

Price: 25.00

Category: Electronics

This validates that:

1. The API successfully persisted the item.
2. The SKU is stored correctly.
3. The quantity and price were stored correctly.
4. The item is associated with the correct category.
5. The database state matches the API request.

Assumption: The table and column names are based on the assessment information and should be adjusted according to the actual database schema.

Assessment Requirement 2 — CI/CD Integration

Requirement

Explain how the automation tests would be integrated into a CI/CD pipeline using GitHub Actions, GitLab, or Jenkins.

Proposed Solution — GitHub Actions

I would use **GitHub Actions** to execute the automation suite as part of the CI/CD pipeline.

Pipeline Flow

```
graph TD; A[Code Change] --> B[Pull Request]; B --> C[Checkout Repository]; C --> D[Setup Java]; D --> E[Install Dependencies]; E --> F[ ];
```

Run API Tests
↓
Database Validation
↓
Run UI Tests
↓
Generate Test Reports
↓
Upload Test Artifacts

Example GitHub Actions Workflow

```
name: QA Automation Tests

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

jobs:
  automation-tests:

    runs-on: ubuntu-latest

    steps:

      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'

      - name: Install dependencies
        run: mvn clean install -DskipTests
```

- name: Run API tests
run: mvn test -Dtest=ApiTestSuite
- name: Run UI tests
run: mvn test -Dtest=UiTestSuite
- name: Upload test results
if: always()
uses: actions/upload-artifact@v4
with:
 name: test-results
 path: target/

Secrets and Environment Configuration

Sensitive information should never be hardcoded in the repository.

Examples include:

```
API_BASE_URL  
DB_URL  
DB_USERNAME  
DB_PASSWORD  
AUTH_USERNAME  
AUTH_PASSWORD
```

These values should be stored securely using **GitHub Actions Secrets** or environment variables.

For example:

```
env:  
  API_BASE_URL: ${ secrets.API_BASE_URL }
```

Assessment Requirement 3 — Test Execution Strategy

Requirement

Explain when API tests and UI tests should be executed to optimize build time and provide fast feedback.

Proposed Execution Strategy

I would use a **layered execution strategy**, prioritizing fast API tests before slower UI tests.

Pull Request

Pull Request



API Smoke Tests



Database Validation

This provides rapid feedback on backend functionality before spending additional execution time on UI tests.

Main Branch

Main Branch



API Regression



Database Validation



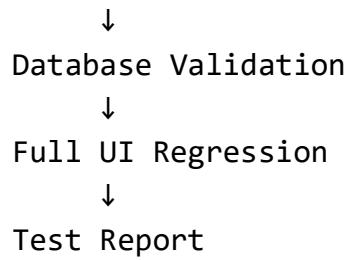
UI Regression

Scheduled Regression

Nightly Run



Full API Regression



Why API Tests Run Before UI Tests

API tests are generally faster and less dependent on browser rendering, UI synchronization, and dynamic page behavior.

They can quickly validate:

- Authentication.
- Business rules.
- Request validation.
- CRUD operations.
- Response status codes.
- Response schemas.
- Backend integrations.

Therefore, API tests are suitable for providing early feedback during development.

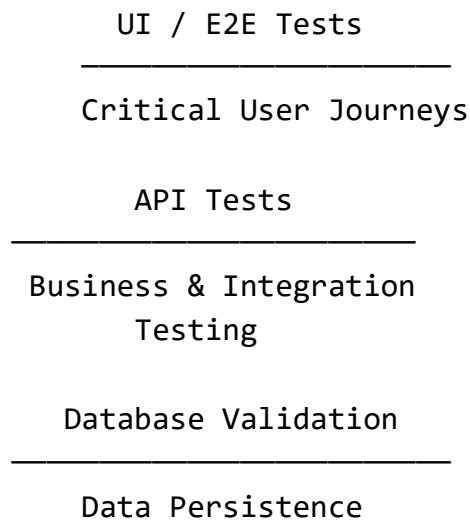
Role of UI Tests

UI automation should focus on critical end-to-end user journeys, such as:

- Store Admin login.
- Product creation.
- Inventory management.
- Order processing.
- Checkout workflows.
- Critical user-facing validations.

UI tests should complement API coverage rather than duplicate all API-level scenarios.

Final Recommended Test Pyramid



This layered approach provides faster feedback, reduces unnecessary UI execution, and makes failures easier to isolate.

Requirement-to-Solution Summary

Database Validation

The API-created inventory item is validated directly against the database using a join between products and categories.

CI/CD Integration

GitHub Actions is used to automate the execution of API tests, database validation, UI tests, and test-result collection.

Execution Strategy

API smoke tests run first to provide fast feedback. Broader API and database validation are followed by UI regression testing, with full regression executed on the main branch or on a scheduled basis.

This approach balances **test coverage, execution speed, stability, and fast feedback** while supporting the scalability requirements of a multi-tenant cloud-based POS application.

Conclusion / Summary

The proposed QA strategy combines UI automation, API testing, database validation, and CI/CD integration to provide reliable and maintainable test coverage for the Electro Pi POS and inventory management platform.

Playwright with Java and the Page Object Model provides a scalable approach for automating critical user journeys while reducing test flakiness through built-in waiting and reliable element synchronization.

API testing provides faster and broader coverage of backend functionality, including request validation, authentication, business rules, and response verification. Database validation complements API testing by confirming that successfully created inventory data is correctly persisted and associated with the expected category.

The CI/CD strategy uses GitHub Actions to execute fast API and database validations earlier in the pipeline, followed by UI tests for critical end-to-end workflows. This layered approach follows the Test Pyramid principle and helps provide faster feedback while maintaining coverage and system confidence.

Because no live application environment or database schema was provided with the assessment, the URLs, selectors, authentication details, API error codes, and database structures used in this submission are reasonable assumptions and should be confirmed or replaced when the actual environment becomes available.